

RAPPORT DE PROJET

CV Search

Application de Sélection Automatisée de Candidatures

**Mini-Projet MIF01
Gestion de Projet et Génie Logiciel**

Réalisé par
BENAOUDIA Ilicia
GHILAS Leticia

Encadré par
Matthieu MOY
Lionel MEDINI
Ryan HORACHE

**Université Claude Bernard Lyon 1 Master 1
Informatique
Année Universitaire 2024-2025**

Table des matières

1. Présentation Globale du Projet	1
2. Design Patterns	1
2.1 Pattern MVC (Modèle-Vue-Contrôleur)	1
2.2 Pattern Observer	3
2.3 Pattern Singleton	3
2.4 Pattern Strategy	4
2.5 Pattern Factory	6
2.6 Pattern Builder	6
2.7 Pattern Facade	7
2.8 Principes SOLID appliqués	8
3. Éthique et Sélection de Candidats	8
3.1 Risques de discrimination	8
3.2 Mesures de mitigation	9
3.3 Limites de l'outil	9
3.4 Recommandations	10
4. Tests	10
4.1 Tests automatiques	10
4.2 Tests manuels effectués	11
5. Conclusion	12

1. Présentation Globale du Projet

CV Search est une application de sélection automatisée de candidatures développée dans le cadre du cours MIF01. L'objectif principal est de permettre à un recruteur de filtrer efficacement des CVs en fonction de compétences techniques recherchées, en appliquant diverses stratégies de sélection.

Fonctionnalités principales

- **Chargement des CVs** depuis des fichiers YAML structurés et import PDF
- **Recherche multi-critères** par compétences avec plusieurs stratégies de filtrage
- **Scoring intelligent** combinant niveau de compétence (70%) et années d'expérience (30%)
- **Gestion d'une shortlist** permettant de marquer les candidats favoris avec notes et commentaires
- **Export des résultats** aux formats CSV et JSON
- **Interface graphique JavaFX** avec synchronisation automatique de vues multiples
- **Import de CV PDF** avec extraction automatique des compétences

L'application a été développée en suivant une approche rigoureuse d'ingénierie logicielle, en appliquant les principes SOLID, les design patterns appropriés, et en mettant en place une couverture de tests exhaustive (52 tests unitaires)

2. Design Patterns

Cette section présente les choix d'architecture et les patterns appliqués dans le projet, en expliquant leur motivation et leur mise en œuvre spécifique.

2.1 Pattern MVC (Modèle-Vue-Contrôleur)

Motivation : Le code initial fourni mélangeait la logique métier et l'interface graphique, rendant le code difficile à maintenir et à tester. Le pattern MVC permet de séparer clairement les responsabilités et de rendre l'application évolutive.

Mise en œuvre :

Le Modèle

Contient toute la logique métier :

- **Applicant** : représente un candidat avec ses compétences et expériences
- **ApplicantList** : gère la collection de candidats et implémente le pattern Observer

- Expérience : encapsule une expérience professionnelle
- Shortlist : gère la liste des candidats favoris (Singleton + Observer)
- PdfCvImporter : importe et parse des CV au format PDF

La Vue

Responsable de l'affichage sans logique métier :

- JfxView : interface graphique principale avec widgets JavaFX
- ShortlistView : fenêtre dédiée à la gestion des favoris

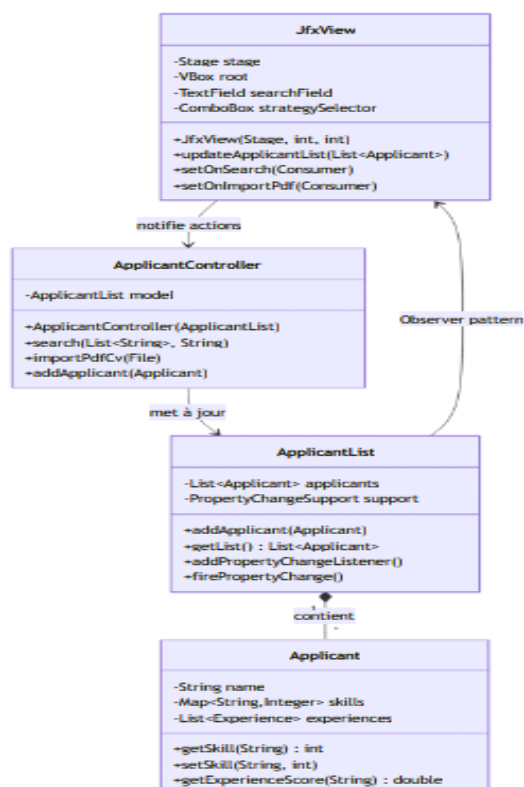
Les vues s'abonnent aux changements du modèle via le pattern Observer et se mettent à jour automatiquement.

Le Contrôleur

Orchestre les interactions entre modèle et vue :

- ApplicantController : traite les actions utilisateur (recherche, ajout de candidat, import PDF), applique les stratégies de filtrage, calcule les scores, et met à jour le modèle

Diagramme de classes :



Avantages constatés :

- **Testabilité améliorée** : le modèle peut être testé sans l'interface graphique
- **Réutilisabilité** : plusieurs vues peuvent afficher le même modèle
- **Maintenabilité** : les modifications d'interface n'impactent pas la logique métier

2.2 Pattern Observer

Motivation : Permettre la synchronisation automatique de multiples vues affichant les mêmes données, sans couplage fort entre les composants.

Mise en œuvre :

- ApplicantList utilise PropertyChangeSupport de Java pour notifier les écouteurs
- Shortlist implémente également Observer pour gérer les mises à jour de la liste de favoris
- Les vues s'enregistrent comme écouteurs via addPropertyChangeListener()

Démonstration : Deux fenêtres JavaFX de tailles différentes affichent l'application. Toute modification dans l'une (ajout de compétence, recherche) est immédiatement répercutée dans l'autre.

```
// Dans App.java
JfxView view1 = new JfxView(stage, 600, 600);
JfxView view2 = new JfxView(new Stage(), 400, 400);

model.addPropertyChangeListener(evt -> {
    view1.updateApplicantList(model.getList());
    view2.updateApplicantList(model.getList());
});
```

2.3 Pattern Singleton

Motivation : La shortlist (liste de favoris) doit être unique dans toute l'application et accessible globalement.

Mise en œuvre :

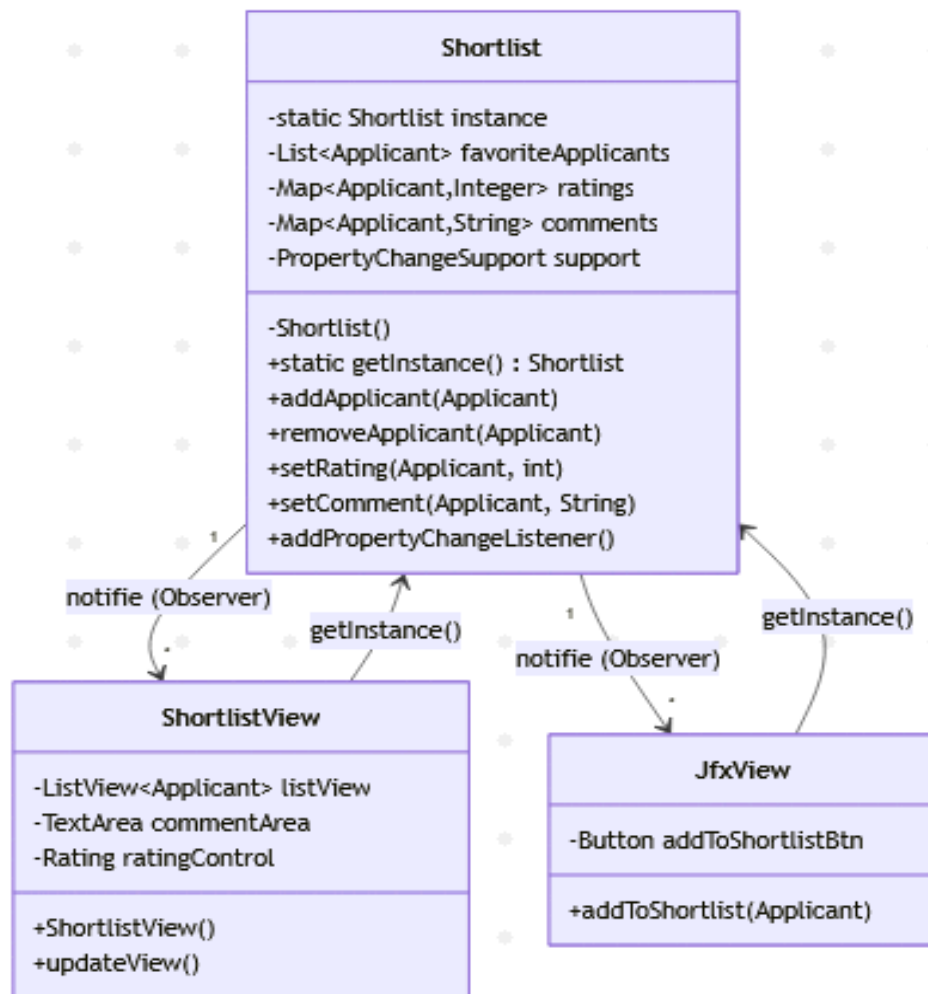
```
public class Shortlist {
    private static Shortlist instance;

    private Shortlist() { }

    public static Shortlist getInstance() {
        if (instance == null) {
            instance = new Shortlist();
        }
        return instance;
    }
}
```

}

Diagramme de classes :



Avantages :

- Une seule instance partagée par toutes les vues
- État centralisé des candidats favoris
- Synchronisation automatique des vues via Observer.

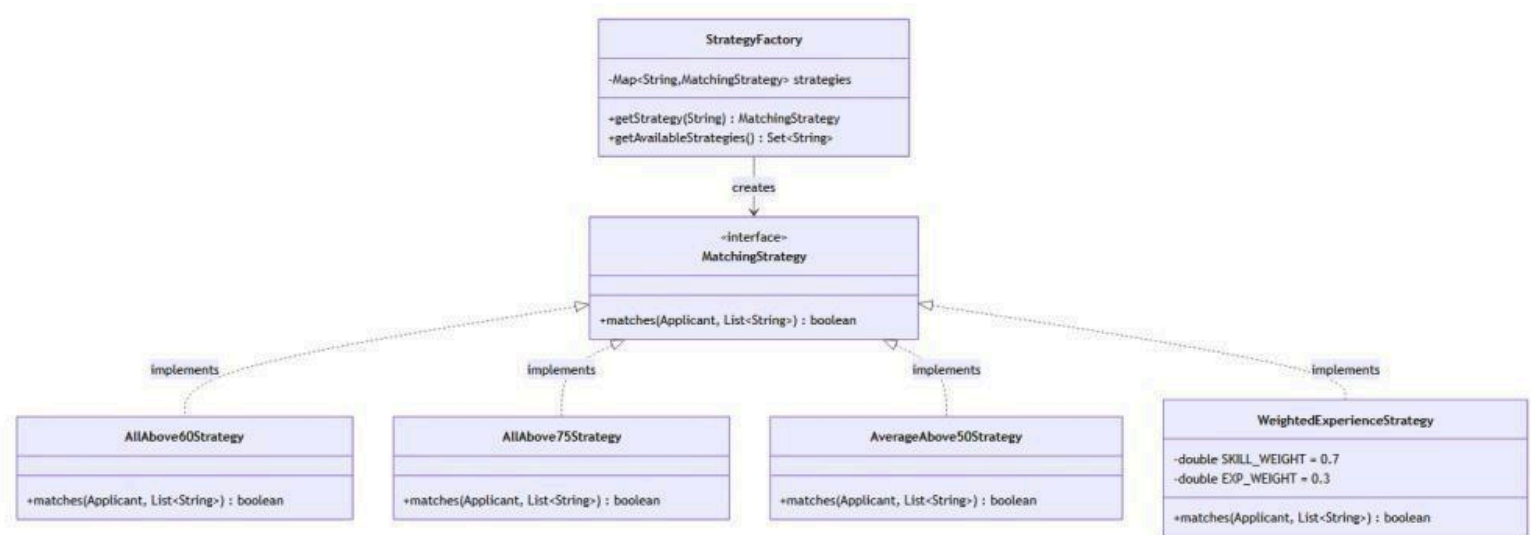
2.4 Pattern Strategy

Motivation : Permettre de changer dynamiquement l'algorithme de sélection des candidats sans modifier le code du contrôleur. Respecte le principe Open/Closed (ouvert à l'extension, fermé à la modification).

Mise en œuvre :

Nous avons créé une interface `MatchingStrategy` implémentée par quatre stratégies concrètes. La classe `StrategyFactory` centralise leur instantiation, permettant au contrôleur de sélectionner dynamiquement la stratégie sans couplage fort.

Diagramme de classes :



Stratégies implémentées :

1. **AllAbove60Strategy** : toutes les compétences $\geq 60\%$
2. **AllAbove75Strategy** : toutes les compétences $\geq 75\%$ (plus strict)
3. **AverageAbove50Strategy** : moyenne des compétences $\geq 50\%$
4. **WeightedExperienceStrategy** : score combiné (70% compétences + 30% expérience) $\geq 50\%$

```
public interface MatchingStrategy {
    boolean matches(Applicant applicant, List<String> skills);
}
```

// Exemple : *WeightedExperienceStrategy*

```
public class WeightedExperienceStrategy implements MatchingStrategy {
    @Override
    public boolean matches(Applicant a, List<String> skills) {
        double totalScore = 0;
        for (String skill : skills) {
            double skillScore = a.getSkill(skill) * 0.7;
            double expScore = a.getExperienceScore(skill) * 100 * 0.3;
            totalScore += (skillScore + expScore) / skills.size();
        }
        return totalScore >= 50;
    }
}
```

2.5 Pattern Factory

Motivation : Centraliser la création des stratégies de matching et permettre l'ajout de nouvelles stratégies sans modifier le code existant.

Mise en œuvre :

```
public class StrategyFactory {
    private static final Map<String, MatchingStrategy> strategies = Map.of(
        "Tout >= 60%", new AllAbove60Strategy(),
        "Tout >= 75%", new AllAbove75Strategy(),
        "Average >= 50%", new AverageAbove50Strategy(),
        "Weighted Experience", new WeightedExperienceStrategy()
    );

    public static MatchingStrategy getStrategy(String name) {
        return strategies.getOrDefault(name, new AllAbove60Strategy());
    }

    public static Set<String> getAvailableStrategies() {
        return strategies.keySet();
    }
}
```

Avantages : - Le contrôleur n'a plus besoin de connaître les classes concrètes - Ajout de nouvelles stratégies sans modifier le code existant - Liste des stratégies disponibles centralisée

Code avant/après :

```
// AVANT (switch/case dans ApplicantController)
switch (strategy) {
    case "Tout >= 60%":
        return new AllAbove60Strategy().matches(a, skills);
    case "Average >= 50%":
        return new AverageAbove50Strategy().matches(a, skills);
    // ...
}
```

```
// APRÈS (utilisation de La Factory)
MatchingStrategy strategy = StrategyFactory.getStrategy(strategyName);
return strategy.matches(a, skills);
```

2.6 Pattern Builder

Motivation : Simplifier la construction d'objets complexes comme ApplicantList à partir de fichiers YAML.

Mise en œuvre :

```
public class ApplicantListBuilder {
    private File directory;
```



```

public ApplicantListBuilder(File directory) {
    this.directory = directory;
}

public ApplicantList build() {
    ApplicantList list = new ApplicantList();
    // Parse tous les fichiers YAML
    // Construit les objets Applicant
    return list;
}
}

```

Utilisation dans PdfCvImporter : Le pattern Builder est également utilisé pour construire un objet Applicant depuis un fichier PDF :

```

public class PdfCvImporter {
    public Applicant importFromPdf(File pdfFile) throws IOException {
        String text = extractTextFromPdf(pdfFile);

        Applicant applicant = new Applicant();
        applicant.setName(extractName(text, pdfFile));

        Map<String, Integer> skills = extractSkills(text);
        for (Map.Entry<String, Integer> entry : skills.entrySet()) {
            applicant.setSkill(entry.getKey(), entry.getValue());
        }

        return applicant;
    }
}

```

2.7 Pattern Facade

Motivation : Simplifier l'export des résultats en masquant la complexité des différents formats d'export.

Mise en œuvre :

```

public class ExportService {
    private static ExportService instance;

    public static ExportService getInstance() {
        if (instance == null) {
            instance = new ExportService();
        }
        return instance;
    }

    public void exportToFile(List<Applicant> applicants,
                            File file,
                            ExportStrategy strategy) throws IOException {
        strategy.export(applicants, file);
    }
}

```

```
}  
}
```

Stratégies d'export :

- CsvExportStrategy : export au format CSV
- JsonExportStrategy : export au format JSON

2.8 Principes SOLID appliqués

Single Responsibility Principle :

- Chaque classe a une responsabilité unique et bien définie
- Applicant : gestion d'un candidat
- Expérience : gestion d'une expérience
- PdfCvImporter : import de CV PDF

Open/Closed Principle :

- Ajout de nouvelles stratégies sans modifier le code existant
- Utilisation de StrategyFactory pour l'extensibilité

Liskov Substitution Principle :

- Toutes les implémentations de MatchingStrategy sont interchangeables

Interface Segregation Principle :

- Interfaces minimales et spécialisées (MatchingStrategy, ExportStrategy)

Dependency Inversion Principle :

- Le contrôleur dépend de l'interface MatchingStrategy, pas des implémentations concrètes

3. Éthique et Sélection de Candidats

3.1 Risques de discrimination

L'utilisation d'un système automatisé de sélection de CV soulève plusieurs questions éthiques importantes :

Biais algorithmiques : - Notre système se base uniquement sur des compétences techniques explicites - Risque de pénaliser les candidats ayant des compétences acquises par l'expérience mais non formalisées - Les candidats autodidactes peuvent être désavantagés par rapport aux diplômés

Exemple LinkedIn (2018) : L'algorithme de recommandations d'offres d'emploi de LinkedIn montrait 20% moins d'offres d'emplois senior bien rémunérés aux femmes qu'aux hommes ayant des compétences équivalentes. L'algorithme se basait sur les taux de clics historiques : comme les femmes cliquaient historiquement moins sur ces offres (par manque de confiance ou discrimination perçue), l'algorithme leur en montrait encore moins, créant un cercle vicieux. Ce cas illustre comment un algorithme peut amplifier les inégalités existantes même sans intention discriminatoire.

Source : Economic Journal (2020), "Algorithmic Glass Ceiling in Social Networks"

Discrimination indirecte : - Le scoring basé sur les années d'expérience (30%) peut défavoriser les jeunes diplômés - L'absence de prise en compte de la formation continue peut pénaliser certains profils

3.2 Mesures de mitigation

Transparence du scoring : - Affichage du score détaillé pour chaque candidat - Décomposition visible : compétences (70%) + expérience (30%)

Flexibilité des stratégies : - Quatre stratégies différentes permettent d'adapter le filtrage selon le contexte - La stratégie "Average >= 50%" favorise les profils polyvalents - La stratégie "Weighted Expérience" valorise l'expérience pratique

Import PDF : - L'ajout de l'import PDF permet de traiter des CV non structurés - Extraction intelligente des compétences avec scoring adaptatif selon le niveau mentionné (expert, avancé)

3.3 Limites de l'outil

Cas où un humain ferait mieux :

1. **Compétences transversales :** notre outil ne détecte pas le leadership, la communication, le travail d'équipe
2. **Projets personnels :** un candidat avec des projets GitHub impressionnants mais peu d'expérience formelle sera sous-évalué
3. **Reconversion professionnelle :** les candidats changeant de domaine sont pénalisés par le manque d'expérience directe
4. **Contexte international :** les diplômes et expériences étrangers ne sont pas pondérés correctement

Exemple concret : Un candidat ayant 2 ans d'expérience en Java avec un niveau expert (score 95) obtiendrait : - Score compétence : $95 \times 0.7 = 66.5$ - Score expérience : $2 \times 10 \times 0.3 = 6$ - Score total : 72.5

Un candidat avec 5 ans d'expérience et niveau moyen (score 70) obtiendrait : - Score compétence : $70 \times 0.7 = 49$ - Score expérience : $5 \times 10 \times 0.3 = 15$ - Score total : 64

Le système favorise la compétence technique sur l'ancienneté, ce qui peut être pertinent pour des postes techniques, mais moins pour des postes de management.

3.4 Recommandations

Pour un usage en production : 1. **Ne jamais utiliser l'outil seul** : toujours faire valider par un recruteur humain 2. **Auditer régulièrement** les résultats pour détecter des biais systématiques 3. **Permettre aux candidats** de contester le scoring algorithmique 4. **Documenter clairement** les critères de sélection auprès des candidats

4. Tests

4.1 Tests automatiques

Le projet contient **52 tests unitaires** répartis comme suit :

Tests du modèle :

- ApplicantTest (4 tests) : parsing YAML, gestion des compétences, cas limites (null, chaînes vides)
- ExperienceTest (3 tests) : calcul de durée, extraction de mots-clés, durée nulle
- ShortlistTest (12 tests) : Singleton, Observer, notes, commentaires, cas limites

Tests des stratégies :

- WeightedExperienceStrategyTest (7 tests) : scoring pondéré, seuils, cas limites
- Tests pour chaque stratégie de matching

Tests d'export :

- ExportStrategyTest (6 tests) : export CSV et JSON, vérification du contenu

Tests du contrôleur :

- ApplicantControllerTest (8 tests) : recherche, application des stratégies, seuils

Tests d'import PDF :

- PdfCvImporterTest (6 tests) :
- Extraction du nom depuis le texte
- Extraction des compétences techniques
- Gestion des fichiers inexistants ou null
- Utilisation du nom de fichier en fallback
- Scoring adaptatif selon le niveau d'expertise (expert, avancé)

4.2 Tests manuels effectués

Test 1 : Synchronisation des vues

- Action : Lancer l'application avec deux fenêtres (600×600 et 400×400)
- Opérations : Ajouter une compétence "Java" dans la première fenêtre
- Résultat attendu : La compétence apparaît immédiatement dans les deux fenêtres
- ✓ **Succès**

Test 2 : Import PDF

- Action : Créer un CV PDF contenant "Jean Dupont" et les compétences "Java, Python, Docker".
- Opération : Cliquer sur "Import PDF" et sélectionner le fichier .
- Résultat attendu : Le candidat est ajouté avec les compétences détectées.
- ✓ **Succès** : Les compétences sont extraites avec des scores adaptés (60-95 selon les mentions).

Test 3 : Export CSV et JSON

- Action : Effectuer une recherche avec 3 candidats correspondants .
- Opérations : Exporter en CSV puis en JSON .
- Résultat attendu : Fichiers correctement formatés avec toutes les informations .
- ✓ **Succès.**

Test 4 : Stratégie Weighted Experience

- Action : Rechercher "Java" avec la stratégie "Weighted Experience"
- Candidat 1 : Java=95, 2 ans d'expérience → score 72.5
- Candidat 2: Java=70, 5 ans d'expérience → score 64
- Résultat attendu : Candidat 1 classé avant Candidat 2
- ✓ **Succès**

Test 5 : Shortlist avec notes et commentaires

- Action : Ajouter 3 candidats à la shortlist
- Opérations : Attribuer des notes (3, 4, 5 étoiles) et des commentaires
- Résultat attendu : Tri automatique par note décroissante, persistance des commentaires
- ✓ **Succès**

Test 6 : Gestion des erreurs

- Action : Tenter d'importer un fichier PDF corrompu
- Résultat attendu : Message d'erreur approprié sans crash
- ✓ **Succès**

5. Conclusion

Le projet CV Search démontre l'application concrète des principes d'ingénierie logicielle. L'architecture MVC combinée avec les patterns Strategy, Factory, Singleton, Builder, Observer et Façade permet d'obtenir un code maintenable, extensible et testable.

L'ajout de fonctionnalités avancées (import PDF, scoring pondéré, export multi-format) illustre la flexibilité de l'architecture mise en place. Les 52 tests unitaires garantissent la fiabilité du code et facilitent les évolutions futures.

La réflexion éthique souligne l'importance d'utiliser ces outils d'aide à la décision avec discernement, en conservant toujours un contrôle humain dans le processus de recrutement.

Extensions possibles : - Machine Learning pour améliorer la détection de compétences dans les PDF - Import depuis LinkedIn API - Analyse sémantique pour détecter les compétences transversales - Interface web avec Spring Boot

Statistiques finales : - 52 tests unitaires (100% de succès) - 0 violation Checkstyle - Intégration continue fonctionnelle (GitLab CI) - 22 classes Java - 4 stratégies de matching - 2 formats d'export (CSV, JSON) - Import YAML + PDF